

Curso / Kurtsoa	Fecha / Data	Eval. /Ebal.	Calificación Kalifikazioa
2025/26	2025-11-19	1º EVAL	
Módulo / Modulua			Cód. exam. Azter. kod.
Programación de Servicios y Procesos			PSP-E
Contenidos / Edukinak (U. Didácticas, .../ U. Didaktikoak, ...)	UD01-UD02		

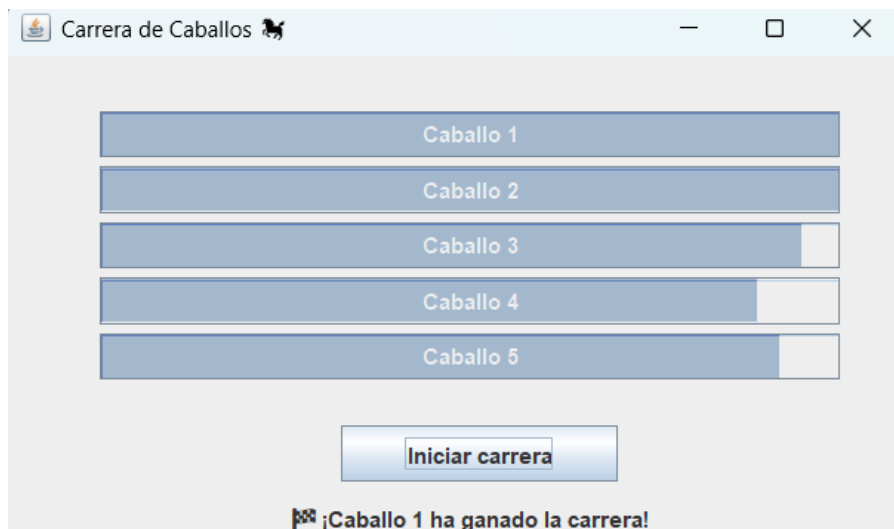
Nombre: _____

EJERCICIO1 - Programación Multithread

RA2-10 puntos

Elabora un programa que simule una carrera de caballos con barras de progreso

Cada caballo se representará con una barra de progreso, que avanzará de forma gradual hasta alcanzar la meta.



La carrera comenzará al pulsar un botón, y los caballos irán avanzando poco a poco hasta que uno de ellos llegue al final.

Cuando uno de los caballos alcance la meta, el programa mostrará un mensaje indicando cuál ha sido el ganador y se detendrá el avance de las barras.

La aplicación debe cumplir los siguientes requisitos:

1. Ser ThreadSafe
2. La ventana principal debe contener:
 - Varias barras de progreso (JProgressBar) con los nombres de los caballos.
 - Un botón “Iniciar carrera”.
 - Un espacio o etiqueta para mostrar el nombre del ganador.
3. Al pulsar el botón, las barras deben comenzar a avanzar hasta que una de ellas llegue al final. El avance será aleatorio y por tanto diferente entre una barra y otra.
4. Cuando una de las barras se complete al 100% o, lo que es lo mismo, llegue al final, el resto de las barras deben detenerse.

5. El programa debe identificar cuál llegó primero y mostrarlo como ganador.
6. La sincronización debe ser de método, tal y como lo hemos visto en clase.

Cómo usar una JProgressBar

Una barra de progreso sirve para representar visualmente un porcentaje de avance entre un mínimo y un máximo.

Por ejemplo:

```
JProgressBar barra = new JProgressBar(0, 100); // mínimo 0, máximo 100
barra.setValue(0);                          // valor inicial
barra.setStringPainted(true);                // muestra el número en la barra

// Simulación de avance
for (int i = 0; i <= 100; i += 10) {
    barra.setValue(i);                      // actualiza el valor
    Thread.sleep(200);                      // pausa breve para ver el efecto
}
```

Cada llamada a `setValue(int)` actualiza la posición de la barra, mostrando el progreso en pantalla.

Para que cada caballo avance de forma aleatoria y desincronizada, haciendo que la carrera sea impredecible podéis utilizar la siguiente formula:

//Avance aleatorio de la barra

```
final int nuevoValor = barra.getValue() + (int)(Math.random()*10);
```

```
barra.setValue(Math.min(nuevoValor, 100));
```

//Para que la carrera sea impredecible

```
Thread.sleep((int)(Math.random()*150 + 50));
```

Utilizar la clase `CarreraCaballosGUI2` que se proporciona en los recursos del examen como base para la resolución del problema.

Elabora un programa Cliente-Servidor en Java que permita gestionar un pequeño ranking de jugadores, donde cada cliente enviará su información y recibirá la lista actualizada de todos los participantes.

La aplicación debe cumplir los siguientes requisitos:

1. Clase Jugador

- Representará a cada participante del ranking.
- Contendrá al menos los siguientes datos:
 - Nombre del jugador
 - Puntuación obtenida
 - Tiempo de conexión
 - El tiempo de conexión se utilizará solo dentro de la aplicación y no debe formar parte de la información intercambiada entre cliente y servidor.
 - El tiempo de conexión será de tipo `java.sql.Timestamp` y se calculará de la siguiente forma:

`Timestamp tiempoConexion = new Timestamp(System.currentTimeMillis());`

2. Servidor

- Permitirá la conexión de varios clientes, con un máximo de 3.
- El servidor terminará cuando se hayan conectado 3 clientes y se mostrará un mensaje adecuado.
- Cada cliente enviará un objeto con la información de su jugador.
- El servidor almacenará los datos recibidos y enviará al cliente la lista completa de jugadores registrados hasta el momento.
- El servidor deberá mantenerse activo mientras funcione correctamente y finalizar de forma controlada en caso de producirse un error o interrupción.
- Todos los recursos utilizados en la comunicación deberán cerrarse correctamente al finalizar la conexión o al producirse un error.

3. Cliente

- Pedirá al usuario su nombre y su puntuación.
- Enviará la información correspondiente a un objeto de tipo Jugador al servidor.
- Recibirá del servidor la lista de jugadores actualizada y la mostrará por pantalla incluido el tiempo de conexión.
- Todos los recursos utilizados en la comunicación deberán cerrarse correctamente al finalizar la ejecución o si ocurre algún error.
- Cuando el cliente finalice deberá mostrarse el siguiente mensaje "Cliente finalizado".

4. Comportamiento general

- Cada ejecución del cliente representa a un jugador distinto.
- El servidor gestionará correctamente las conexiones y los datos enviados.
- Los datos de los jugadores deberán transmitirse y actualizarse de forma adecuada.